MARQUE

E-Commerce Frontend Capstone — Project Documentation

Repository:

github.com/arathism/Week7_ECommerce-Website

Live Application:

arathism.github.io/Week7_ECommerce-Website

Built with React 19, React Router v7, Context API, and FakeStoreAPI

Table of Contents

1. Project Overview

MARQUE is a catalog-style e-commerce single-page application built as the capstone project for a front-end development internship (Week 7/8 module). It demonstrates the complete set of skills covered across the program: component-based UI architecture, client-side routing, global state management, REST API integration, form validation, performance optimization, and static-site deployment.

1.1 Goals and Objectives

- Recreate the core shopping journey of a real storefront: browse products → view details → manage a shopping bag → complete checkout.
- Cleanly separate presentation (components), state (React Context), and data access (a dedicated services layer) so each concern can evolve independently.
- Integrate a live third-party REST API (FakeStoreAPI) for real product data rather than static mock data.
- Implement a simulated authentication flow with protected routing, backed by browser localStorage since no backend server is part of the assignment scope.
- Apply meaningful performance techniques — route-based code splitting, lazy image loading, and memoized derived state — rather than treating optimization as an afterthought.
- Produce a deployable build with configuration for GitHub Pages, Netlify, and Vercel.

1.2 Target Audience

This documentation is written for graders/reviewers assessing the capstone submission, and for any developer who wants to run, extend, or maintain the codebase going forward.

2. Setup Instructions

Prerequisites: Node.js 18 or later, and npm (bundled with Node).

2.1 Local Installation

```
# 1. Clone the repository
git clone https://github.com/arathism/Week7_ECommerce-Website.git
cd Week7_ECommerce-Website

# 2. Install dependencies
npm install

# 3. Start the development server
npm run dev
# App runs at http://localhost:5173
```

2.2 Testing, Building, and Previewing

```
# Run the unit test suite
npm test

# Build an optimized production bundle (outputs to /dist)
npm run build
```

```
# Preview the production build locally
npm run preview
```

No environment variables, API keys, or backend server are required — the application communicates directly with the public FakeStoreAPI service over HTTPS.

3. Code Structure

The project follows a feature-and-responsibility folder layout under src/, keeping UI, state, data access, and styling in clearly separated locations:

```
src/
  App.js              Route definitions, lazy-loaded pages, provider wiring
  App.css             App-shell styles
  main.jsx            Entry point — mounts Router + AuthProvider + CartProvider

  components/
    Navbar/            Sticky nav: branding, links, cart badge, auth state
    ProductList/       Filter/sort controls + responsive product grid
    ProductCard/       Individual catalog card
    Cart/              Bag line items, quantity controls, subtotal
    Checkout/          Shipping + payment form, validation, order summary
    Loading/           Reusable loading indicator
    ErrorBoundary/     Catches render errors, shows a recovery screen
    ProtectedRoute.js  Redirects to /login if a route requires auth

  pages/
    Home.js             Hero + ProductList, order-confirmation banner
    ProductDetail.js    Single product fetch + add-to-bag
    CartPage.js         Wraps <Cart />
    CheckoutPage.js     Wraps <Checkout />, route-protected
    Login.js/Register.js  Simulated auth forms
    NotFound.js         404 fallback

  contexts/
    CartContext.js      Cart state (useReducer) + localStorage persistence
    AuthContext.js      Simulated auth (register/login/logout) via localStorage

  hooks/
    useProducts.js      Fetches products + categories, exposes filter/sort state

  services/
    api.js              All fetch() calls to FakeStoreAPI live here

  styles/
    variables.css       Design tokens (color, type, spacing)
    global.css          Resets + shared utility classes
```

3.1 Organizing Principle

Components own their own markup and co-located stylesheets; contexts and hooks own state and data logic; services/api.js is the single file aware of the network. As a result, no component calls fetch() directly, and replacing or extending the backend later only touches one file.

4. Technical Details & Architecture Decisions

4.1 Technology Choices

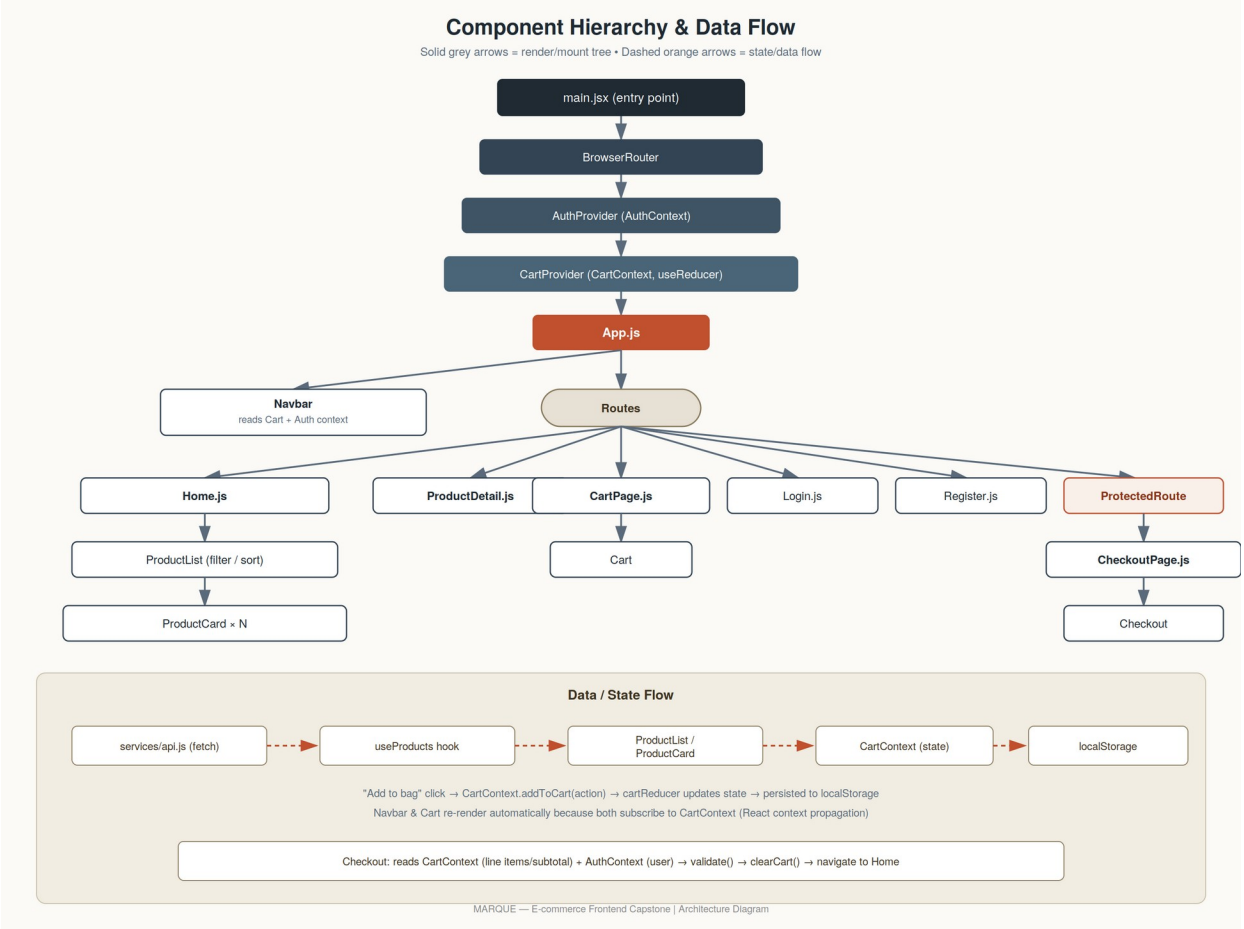
- Vite + React 19 — fast dev server and native ES module bundling for build performance.
- React Router v7 (react-router-dom) — handles all client-side routing, including a protected /checkout route.
- Context API over Redux — the app has exactly two pieces of cross-cutting state (cart and auth), each with a small, well-defined set of actions. A useReducer inside CartContext provides Redux-style predictability via a single, independently unit-tested cartReducer function, without adding an extra dependency.
- Local component state (useState) is reserved for anything that does not need to be shared across components: form fields, transient “just added” button feedback, etc. Filter/sort selection lives inside the useProducts hook, since only ProductList and Home consume it.
- Simulated authentication, exactly as scoped by the assignment: AuthContext stores registered users and the active session in localStorage. There is no real backend or password hashing — this is a deliberate front-end-only demonstration of protected routing and auth-aware UI, not a production authentication system.

4.2 Core Algorithms & Data Structures

- Cart reducer (finite-state reducer pattern): cart state is an array of { id, quantity, ...product } line items. Actions (ADD_ITEM, REMOVE_ITEM, INCREMENT, DECREMENT, CLEAR) are pure functions of (state, action) → newState, which is what makes the reducer trivial to unit test in isolation from any UI.
- Client-side filter/sort pipeline: the full product list is fetched once, then filtering (by category) and sorting (by price or rating) are derived values computed with useMemo, so the expensive array operations only re-run when their dependencies (the filter/sort selection) actually change — not on every render.
- Form validation: the checkout validate() function runs a set of field-level pure predicate checks (required fields, postal code pattern, card number format via regex + Luhn-style spacing tolerance, expiry pattern, CVV length) and returns an errors object keyed by field name, which the form consumes to render inline messages and gate the submit button.
- Persistence layer: both CartContext and AuthContext synchronize their in-memory state to localStorage via a useEffect keyed on the relevant state slice, giving simple write-through persistence without a database.

5. Component Architecture

The diagram below shows the full render/mount hierarchy (solid grey arrows) starting from the application entry point down to individual route components, alongside the runtime data-flow path for adding an item to the cart (dashed orange arrows, lower panel).



Read the diagram top to bottom for the render tree (solid arrows): providers wrap the router, App.js renders the Navbar plus the active Route, and each route mounts its own page/component subtree. The lower panel shows the runtime data path when a user adds a product to the bag: the click handler calls into CartContext, the reducer computes new state, that state is written to localStorage, and every component subscribed to CartContext (Navbar's badge, the Cart page) re-renders automatically.

6. State Management Approach

State is scoped as narrowly as possible: only truly cross-cutting concerns (cart, auth) live in Context; everything else stays local to the component or hook that owns it.

State	Where It Lives	Reasoning
Cart line items	CartContext (useReducer)	Needed by Navbar, Cart, and Checkout — genuinely global state.
Auth session	AuthContext	Needed by Navbar, ProtectedRoute, and Checkout.
Product list + filters/sort	useProducts hook (used in Home)	Local to a single screen; no need to lift it higher.

State	Where It Lives	Reasoning
Form fields (Checkout/Login/Register)	useState in the component	Never read outside that form.
“Added to bag” button feedback	useState in ProductCard	Purely presentational, per-card state.

Both cart and authentication state are persisted to localStorage, so refreshing the page does not empty the shopping bag or sign the user out.

7. API Integration Details

All network requests are centralized in src/services/api.js, a thin wrapper around the native fetch API, which talks to the public FakeStoreAPI:

- getProducts() → GET /products
- getProduct(id) → GET /products/:id
- getCategories() → GET /products/categories
- getProductsByCategory(category) → GET /products/category/:category

The useProducts hook fetches the full catalog and category list once on mount using Promise.all, then performs filtering and sorting client-side (see §4.2 and §8). ProductDetail fetches a single product on demand, keyed off the route's :id parameter, and manages its own loading/error state independently.

Every API call is wrapped in a try/catch block; network failures and non-200 responses surface a readable, user-facing message rather than a blank or broken screen.

8. Performance Optimizations

- Route-level code splitting via React.lazy + Suspense in App.js — each page ships as its own bundle chunk, confirmed in the production build output (separate Home, ProductDetail, CartPage, CheckoutPage, Login, Register, and NotFound chunks).
- Lazy image loading — loading="lazy" is applied to all product images across the catalog grid and cart thumbnails.
- Client-side filtering and sorting with useMemo — avoids re-fetching from the API or recomputing derived lists on every render.
- CSS mix-blend-mode: multiply is used to blend FakeStoreAPI's white-background product photography naturally onto the page background, at zero runtime JavaScript cost (no client-side image processing).
- Reduced-motion support — all animation is disabled under the prefers-reduced-motion: reduce media query for accessibility and to avoid unnecessary paint work.

9. Testing Evidence

Unit tests are written with Vitest and target the two most logic-heavy, side-effect-free units in the codebase: the cart reducer and the checkout form validator.

9.1 Automated Test Coverage

- src/contexts/CartContext.test.js — 6 tests covering cartReducer: adding a new item, incrementing an existing line, removing an item, quantity reaching zero, quantity clamped at zero, and clearing the entire cart.
- src/components/Checkout/Checkout.test.js — 7 tests covering the checkout validate() function: a fully valid form, a missing name, an invalid postal code, a malformed card number, an accepted card number containing spaces, a malformed expiry date, and a too-short CVV.

```
✓ src/contexts/CartContext.test.js (6 tests)
✓ src/components/Checkout/Checkout.test.js (7 tests)

Test Files  2 passed (2)
     Tests  13 passed (13)
```

Run the suite locally with npm test.

9.2 Manual End-to-End Test Cases

- Browse the catalog, filter by category, and sort by price and by rating.
- Open a product detail page from the grid.
- Add an item to the bag from both the grid card and the detail page.
- Adjust and remove bag line-item quantities.
- Register a new simulated account.
- Confirm an unauthenticated visit to /checkout redirects to /login (protected route).
- Submit the checkout form with invalid data and confirm inline validation errors appear.
- Submit the checkout form with valid data and confirm the bag clears and an order-confirmation banner appears back on the Home page.

10. Deployment Steps

The application builds to a static bundle, so it can be hosted on any static file host. Configuration files for the three most common targets are included in the repository.

10.1 GitHub Pages (current live deployment)

GitHub Pages serves project sites from a subpath (username.github.io/repo-name/) rather than the domain root, so two adjustments are required beyond a default Vite build:

- vite.config.js sets base: '/Week7_ECommerce-Website/' to match the exact repository name.
- A postbuild script copies dist/index.html to dist/404.html. GitHub Pages has no server-side rewrite rules, so without this step, refreshing any route other than / (for example /product/3)

would return a 404. Serving the app shell for unknown paths lets React Router take over client-side routing from there.

```
npm install     # picks up the gh-pages package already declared in package.json
npm run deploy   # builds the app, then pushes /dist to the gh-pages branch
```

Then, in the repository settings on GitHub: Settings → Pages → Build and deployment → Source: Deploy from a branch → Branch: gh-pages.

10.2 Netlify (alternative — netlify.toml included)

- Push the repository to GitHub.
- In Netlify: “Add new site” → “Import an existing project” → select the repository.
- Build command: npm run build; publish directory: dist (already configured in netlify.toml, including the SPA redirect rule needed for client-side routing).

10.3 Vercel (alternative — vercel.json included)

- Push the repository to GitHub.
- In Vercel: “Add New Project” → import the repository — the “Vite” framework preset is auto-detected.
- Deploy. vercel.json handles the SPA rewrite so deep routes like /product/3 resolve correctly on a hard refresh.

In all three cases, the key constraint is the same: because routing happens client-side, the host must serve index.html for unrecognized paths — which is exactly what the included configuration files and the GitHub Pages 404.html trick accomplish.

11. Challenges Faced

- JSX inside .js files: the submission structure required component files to use a .js extension rather than .jsx, while still containing JSX syntax. Vite's default toolchain expects .jsx/.tsx for JSX content. This was resolved by explicitly configuring esbuild's loader to 'jsx' for files under src/ in both vite.config.js and vitest.config.js, and pinning @vitejs/plugin-react to a release compatible with the classic (non-Rolldown) Vite pipeline in use.
- Simulated authentication vs. real persistence: with no backend available, both registration and login are implemented against localStorage. This is documented directly in AuthContext.js as a simulation for demonstrating protected routes and auth-aware UI — explicitly not a security pattern intended for reuse in a production system.
- Keeping filters responsive: re-fetching from the API on every filter or sort change would introduce a noticeable lag. Fetching the catalog once and deriving the visible list with useMemo keeps filter and sort interactions instant.
- Product card visual styling: achieving a clean, intentional “perforated notch” edge on the product card component across every grid width took several iterations. The final approach uses two pseudo-elements pinned to the card's edge rather than attempting to clip the card element itself, which avoided rendering glitches at narrow breakpoints.

12. Visual Documentation

The application is live and publicly reachable at the links below; the screenshots that follow were captured directly from the live deployment.

- Live application: https://arathism.github.io/Week7_ECommerce-Website/
- Source repository: https://github.com/arathism/Week7_ECommerce-Website

12.1 Catalog / Home Page

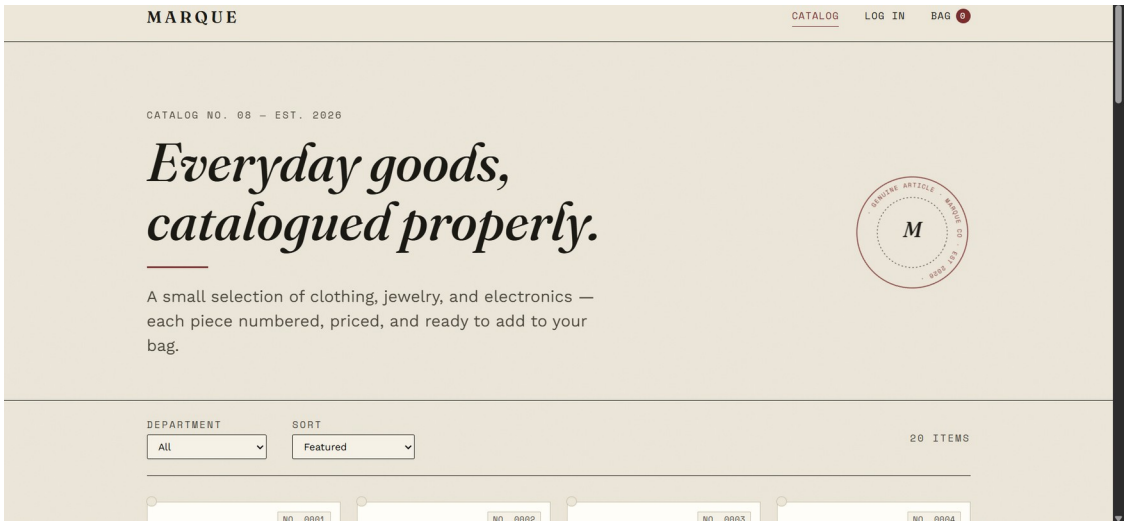


Figure 1 — Home/catalog page hero section with department filter, sort control, and item count.

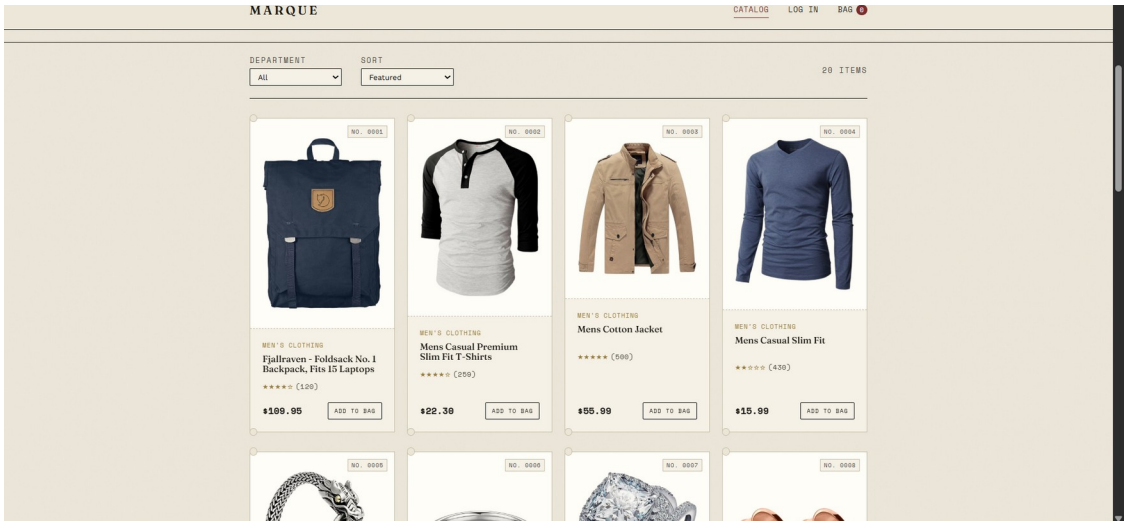


Figure 2 — Product grid showing catalog cards with category, rating, price, and “Add to Bag” action.

12.2 Authentication

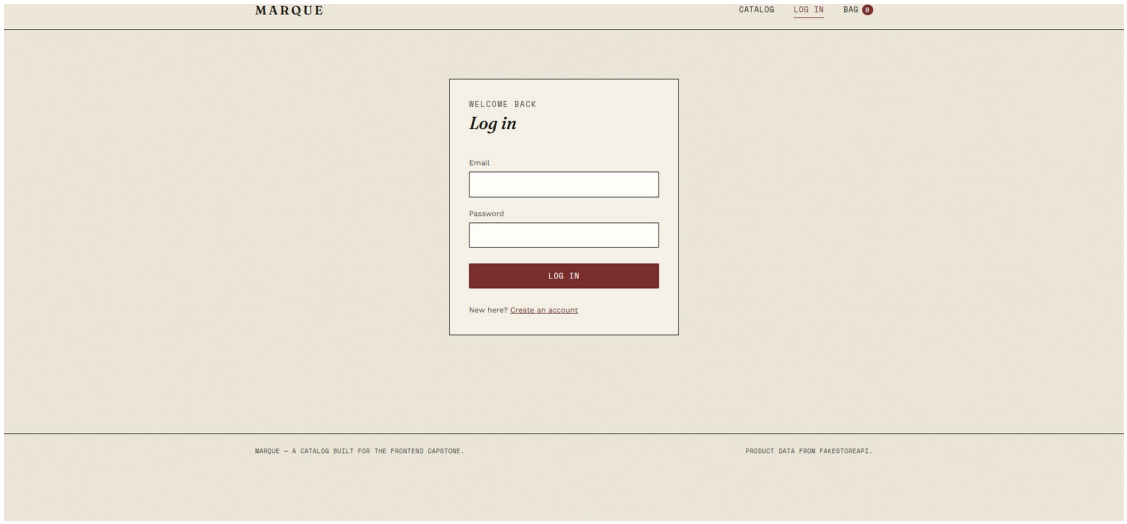


Figure 3 — Login page (simulated auth via AuthContext / localStorage).

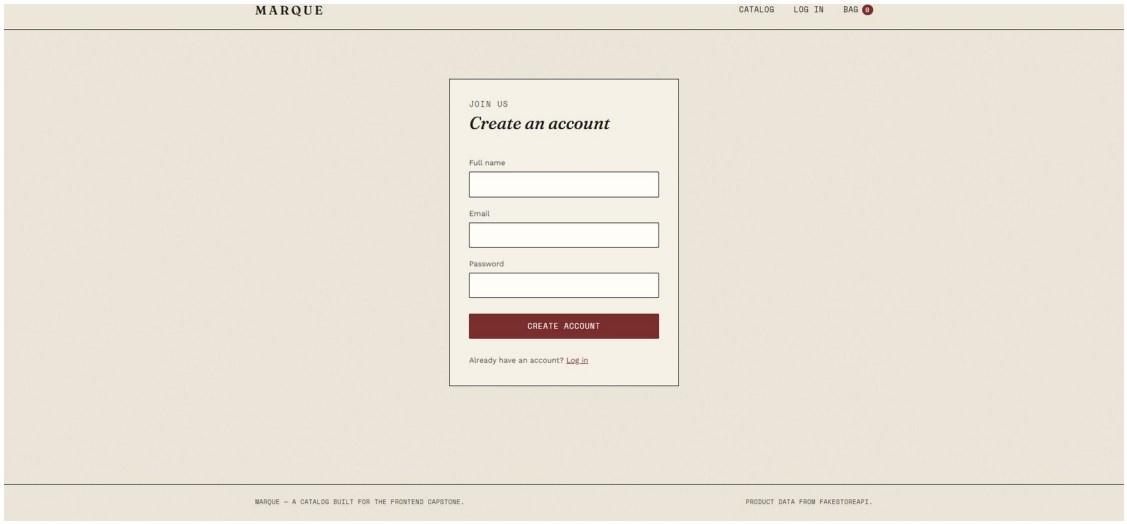


Figure 4 — Registration page for creating a simulated account.

12.3 Shopping Bag

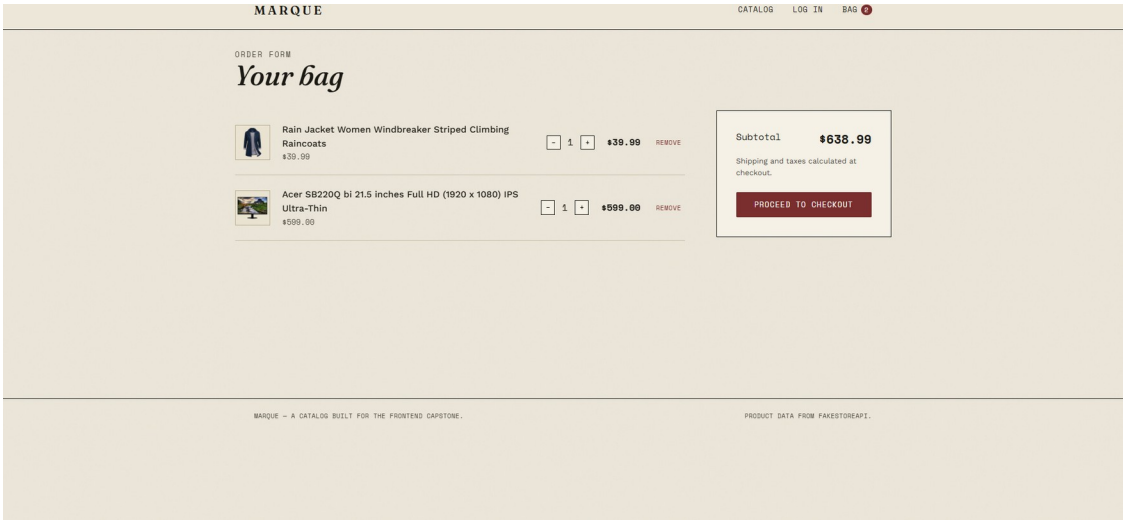


Figure 5 — Shopping bag with two line items, quantity controls, and computed subtotal, ready for checkout.

Additional views worth capturing for a fully exhaustive record: a product detail page, the checkout form (including an inline validation error state), and a narrow-viewport/mobile layout. These can be added to this section the same way if needed.

13. Quality Standards Checklist — Cross-Reference

The table below maps each required documentation item to the section that satisfies it, for quick grading reference.

Requirement	Where It's Covered
Project Overview	Section 1 — goals, objectives, and scope.
Setup Instructions	Section 2 — install, dev, test, build, preview commands.
Code Structure	Section 3 — full file/folder hierarchy with rationale.
Visual Documentation	Section 12 — live URL provided; screenshot checklist and embed instructions included.
Technical Details (algorithms, data structures, architecture)	Section 4 — reducer pattern, memoized filtering, validation logic, persistence layer.
Testing Evidence	Section 9 — 13 automated Vitest cases plus a manual E2E test checklist.
Component Architecture (hierarchy + data flow diagram)	Section 5 — full render tree and cart data-flow diagram.
State Management Approach	Section 6 — state ownership table.
API Integration Details	Section 7 — endpoint list and fetch strategy.

Requirement	Where It's Covered
Performance Optimizations	Section 8 — code splitting, lazy images, memoization.
Deployment Steps	Section 10 — GitHub Pages, Netlify, and Vercel instructions.
Challenges Faced	Section 11 — four concrete technical challenges and their resolutions.